Open Access | https://doi.org/10.25080/yyxu9446

Engineering Better Retrieval for RAG

Aarti Jha[1], and **Mahima Arora**[1]

[1]Red Hat India Pvt. Ltd.

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Abstract

Retrieval-Augmented Generation (RAG) has become the most common approach for grounding large language models in external knowledge, yet most deployments still treat retrieval as a solved problem: naive chunking followed by cosine similarity search. In practice, retrieval quality, not generation capability, determines whether a RAG system produces faithful answers or confident hallucinations. This paper argues that retrieval optimization requires a staged engineering approach and presents a three-part framework covering the pipeline before, during, and after vector search. The framework specifies the retrieval subsystem used in standalone pipelines and as a callable tool within agent workflows. We formalize the failure modes that arise at each stage and describe the techniques practitioners use to address them, from hierarchy-aware chunking and query transformation through reranking and hybrid search to context grading, refinement, and compression before prompt assembly. We extend the framework to multimodal corpora where text, tables, and images demand typed representations and hybrid semantic-deterministic retrieval. Using a curated corpus of research papers and ground-truth question-answer pairs, we evaluate pipeline variants with RAGAS metrics and show that combined optimizations across all three stages yield substantial gains in faithfulness and answer relevancy over naive baselines.

Keywords retrieval-augmented generation, information retrieval, vector databases, large language models, document chunking, RAG evaluation

1. Introduction

Retrieval-Augmented Generation (RAG) [1] grounds large language models (LLMs) in external knowledge at inference time, enabling domain-specific applications without costly retraining. The standard pipeline is well known: load documents, split them into chunks, embed the chunks into a vector database, retrieve the most similar entries at query time, and condition generation on the returned context. Many production systems now wrap this pattern in agent workflows that select among tools and may call retrieval more than once before answering [2], [3]; the retrieval step itself, however, remains the same architectural primitive. Teams that invest heavily in model selection and prompt engineering often discover that retrieval, not generation, is the main constraint on whether the system returns faithful and relevant answers.

The gap between a prototype RAG system and a reliable one almost always appears in retrieval. First, poor chunk boundaries split documents mid-thought; although systems typically retrieve multiple chunks, an answer that spans a poorly placed split may still arrive incomplete because no retrieved chunk contains the full semantic unit [4]. Second, user queries arrive abbreviated, multi-intent, or dependent on prior conversation, causing embedding models to match against irrelevant indexed content [5], [6]. Third, even when retrieval returns genuinely relevant context, loosely related passages in the top-k set weaken the context and increase the burden on the generator to identify what matters [7].

Published Jul 13, 2026

Correspondence to
Aarti Jha
aajha@redhat.com

Open Access

Copyright © 2026 Jha & Arora. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

We characterize these recurring problems as three failure modes. *Context fragmentation* arises when chunk boundaries break apart meaningful sections. *Query mismatch* occurs when the query embedding poorly represents the user's actual information need. *Retrieval noise* describes situations where the retrieved set contains too many redundant or weakly relevant chunks for the generator to use effectively. Switching to a larger LLM or expanding the context window does not resolve any of these; each originates from how documents are prepared, how queries are processed, or how candidates are selected and ranked.

Existing literature [4], [5], [6], [7] and practitioner guides discuss individual remedies such as better chunking, query expansion, and reranking, each addressing specific retrieval challenges. However, these techniques are typically presented and evaluated in isolation, under different assumptions and experimental settings, making it difficult for practitioners to determine when a particular method is appropriate, how it interacts with others, and which retrieval failure it is intended to address. As a result, retrieval optimization is often approached through trial-and-error rather than step-by-step diagnosis. This paper provides a unifying framework that organizes these existing techniques around three stages where retrieval quality can be systematically improved: **Pre-Retrieval**, which prepares documents and queries before search begins; **Mid-Retrieval**, which controls what the search returns and in what order; and **Post-Retrieval**, which grades, refines, and compresses retrieved content before assembling it into the generator prompt or returning it to an orchestrating agent. The framework further connects each stage to the retrieval failures it addresses, emphasizing that problems introduced early in the pipeline cannot be fully corrected downstream—for example, no amount of reranking can recover information that was never indexed, and no post-retrieval filtering can compensate for an ambiguous query that retrieved the wrong documents in the first place.

The contributions of this work are:

1. A three-stage retrieval optimization framework that maps techniques to pipeline stages and to the failure modes they resolve
2. A systematic treatment of pre-retrieval techniques for document chunking, query transformation, and semantic routing
3. A taxonomy of mid-retrieval components spanning diversity optimization, filtering, parent-child expansion, hybrid search, and reranking
4. Post-retrieval components for context grading, refinement, and compression before generation

Figure 1 illustrates the framework. The sections that follow treat each stage in turn, describing the techniques available, the conditions under which they help, and the empirical evidence for their effect on retrieval quality.

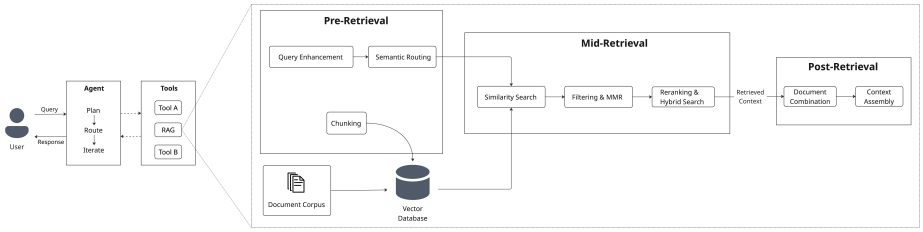


Figure 1. *Three-stage retrieval optimization framework. Pre-Retrieval prepares documents and queries; Mid-Retrieval optimizes search quality and diversity; Post-Retrieval refines retrieved context before prompt assembly. The stages define the retrieval subsystem used in standalone pipelines and as a tool within agent workflows.*

1.1. *Where This Framework Applies*

Standalone RAG executes retrieval once per query and passes the result directly to a generator. Agent-based systems [2] expose retrieval as one tool among others (structured query, web search, APIs), and an orchestrating model decides whether to invoke it, which index or route to query, and whether to call it again when context is insufficient.

Despite these architectural differences, each retrieval invocation passes through the same operational concerns: documents must be indexed well, the query must be prepared, candidates must be ranked and filtered, and the returned context must be packaged for downstream use. The three stages in Figure 1 specify **the retrieval subsystem**: the logic inside the retrieval tool rather than the full agent loop. The agent layer decides *when* to retrieve; the sections that follow detail *what each call should do* at each stage.

2. Related Work

Modern RAG pipelines inherit retrieval primitives from dense and neural ranking research. Dense Passage Retrieval (DPR) [8] established bi-encoder embeddings as a viable alternative to BM25 [9] for open-domain question answering. ColBERT [10] introduced late interaction, pre-computing document token representations while scoring query–document relevance at retrieval time, offering finer-grained matching than bi-encoders without the latency of full cross-encoder encoding. Cross-encoder reranking [11] remains the standard precision stage when joint query–document scoring is affordable. Hybrid sparse–dense fusion and diversity-aware selection address complementary mid-retrieval failure modes. This paper organizes these techniques as composable mid-retrieval components rather than isolated design choices.

Subsequent work shifted attention from *whether* to retrieve toward *when* and *how often*. Self-RAG [12] trains reflection tokens so a model decides when to retrieve and how to critique retrieved evidence. Corrective RAG (CRAG) [13] wraps retrieval in an evaluator–refiner loop grading passage relevance, stripping extraneous content, and falling back to web search when local retrieval fails. Agentic RAG surveys [2] generalize this pattern, exposing retrieval as one tool an orchestrator may invoke zero, one, or many times per query. These architectures primarily govern retrieval scheduling and fallback; they offer limited guidance on staging optimizations within a single retrieval call. Our framework complements this line of work by specifying what each invocation should do across pre-, mid-, and post-retrieval stages: chunk preparation, query transformation, candidate ranking, and context grading, refinement, and assembly.

3. Implementation Details

We evaluate pipeline variants on a curated corpus of 100 arXiv research papers [14] spanning machine learning, statistics, mathematics, and computer science, paired with manually curated and synthetically generated question–answer sets. Table 1 lists the fixed configuration; embeddings are stored in Qdrant [15], a vector database. Each experimental variant changes one pipeline technique at a time with a single retrieval call per query, isolating the retrieval subsystem from agent orchestration. Performance is measured with RAGAS [16] metrics like faithfulness, answer relevancy, context precision, and context recall. All techniques in this paper are implemented in the open-source *Retrieval Playground* toolkit [17], which provides composable pre-, mid-, and post-retrieval modules and tutorial notebooks for reproducing the ablations below.

Table 1. *Fixed experimental configuration.*

Component	Setting

Corpus	100 arXiv papers across five AI/data-science topics
Q&A evaluation set	Synthetic generation
Embeddings	`models/gemini-embedding-001` (Gemini API)
Generation	Gemini 2.5 Flash, temperature = 0.0
Protocol	Single retrieval call per query

4. Pre-Retrieval Optimization

Pre-retrieval optimization operates on the two inputs to any retrieval system: the document corpus and the user query. Errors introduced here propagate through every subsequent stage. A document split at the wrong boundary is indexed incorrectly forever; an ambiguous query may produce an embedding that does not accurately represent the user's intent, leading retrieval toward less relevant documents before any ranking logic can intervene.

4.1. Document Chunking

Chunking determines the atomic units of retrieval. Each chunk is embedded and indexed independently, so the splitting function defines what information is discoverable and at what granularity. Consider a methods section that spans three paragraphs: if chunking splits after the first paragraph, a query about the experimental setup may retrieve the introduction to the methods without the setup details that follow.

We compare four chunking strategies by how they partition documents and what each retrieval unit contains (Table 2). *Recursive* chunking (LangChain's RecursiveCharacterTextSplitter) splits text hierarchically: paragraph, then sentence, then word, using a series of natural-language separators. It respects natural-language boundaries better than fixed-size splits and remains the most common text baseline. *Docling* chunking [18] applies layout-aware hybrid parsing, recovering tables, figures, and hierarchical text blocks from PDF and office formats and supporting typed multimodal chunks rather than a flat text stream. *Parent-child* chunking [19] indexes small child chunks (typically 100–400 tokens) for precise matching while storing links to larger parent units (often 1,000–2,000 tokens); mid-retrieval decides whether to return a matched child or expand to its parent (Table 5). *Contextual* chunking [20] first partitions a document into chunks. For each chunk, an LLM is given the full source document and generates a brief contextual description that explains the chunk's role within the document (e.g., the section topic or surrounding discussion). This description is prepended to the chunk before embedding, improving its representation with document-level context and reducing loss of context during retrieval.

Table 2. *Chunking strategies and their properties.*

Strategy	Retrieval Unit	Preserves Structure	Semantic Coherence
Recursive	Hierarchical natural-language boundaries	Partial	Medium
Docling	Layout-aware typed chunks (text, table, image)	Yes	High
Parent-child	Small child chunks; parent unit returned at query time	Partial (via parent)	High
Contextual	Chunk plus prepended situating context	Partial	High

For multimodal documents, only Docling chunking produces typed units rather than homogeneous text. Text chunks carry section hierarchy through parent heading metadata.

Table chunks retain relational structure alongside natural-language descriptions that make tables discoverable through semantic search. Image chunks pair visual content with vision-language descriptions, allowing text queries to match against diagram and figure content. The pre-retrieval remedy is to preserve native structure at index time; mid-retrieval then combines semantic search with deterministic SQL over tables and description-based retrieval over images.

Table 3. *RAGAS evaluation of chunking strategies on our curated corpus. All other pipeline settings held constant; only the chunking strategy varied.*

Strategy	Answer Relevancy	Faithfulness	Context Precision	Context Recall	Average
Recursive	0.972	0.950	0.923	0.900	0.941
Docling	0.979	0.972	0.917	0.893	0.922
Parent-Child	0.978	0.986	0.922	0.900	0.947
Contextual	0.973	0.961	0.922	0.833	0.922

Table 3 shows that no single strategy leads across all metrics. On this text-centric corpus, recursive and parent-child chunking perform strongest: hierarchical boundaries align well with question–answer semantics, and parent-child additionally supports mid-retrieval expansion when matched snippets need surrounding context. Docling's advantage in preserving multimodal structure (tables, figures, layout) is not reflected here because the current evaluation set is mostly text-based; for multimodal corpora, its typed retrieval units become the practical choice.

4.2. Query Enhancement

User queries often arrive abbreviated, multi-intent, or dependent on prior conversation: phrasing that embeds poorly against an indexed corpus. Query enhancement reformulates input into one or more retrieval-ready queries before vector search. Common query transformation techniques include query expansion, rewriting, decomposition, HyDE [21], step-back prompting [22], and multi-query generation. We focus on the query transformation techniques most commonly used in production RAG pipelines, each targeting a distinct mismatch pattern and applicable independently or in combination.

Table 4. *Query enhancement methods with example transformations.*

Method	Original Query	Enhanced Query
Query expansion	LLM FT best practices?	What are best practices for large language model fine-tuning?
Query decomposition	Compare transformer and CNN architectures for image segmentation and their computational requirements	What are transformer architectures for image segmentation?; What are CNN architectures for image segmentation?; What are their computational requirements?
Query rewriting	What about its performance on ImageNet?	What is ResNet-50's performance on ImageNet?
Self-querying	How do modern RAG systems handle multimodal documents?	What chunking strategies do RAG systems use for tables and images?; How is multimodal content embedded and retrieved?

Query expansion resolves abbreviations and compressed phrasing when intent is clear but vocabulary does not match the index. *Query decomposition* splits compound requests into atomic sub-queries retrieved and merged separately. *Query rewriting* uses conversation

history to make follow-ups standalone. *Self-querying* generates multiple focused search queries when a single rewording cannot cover an open-ended request.

When combining these techniques, we recommend applying decomposition before expansion, since expanding a compound query may blur the semantic boundaries between independent information needs. For example let's consider this query: "Explain Docling's table extraction and compare it with TableFormer."

Decomposing first produces two independent searches:

(1) Explain Docling's table extraction and

(2) Compare Docling with TableFormer.

Expanding each sub-query separately keeps the retrieved context focused. Expanding the original compound query first may introduce unrelated synonyms or concepts across both tasks, making retrieval less precise.

4.3. Semantic Routing

Not every query needs the same retrieval path. Greetings, off-topic requests, and domain-specific questions each benefit from different handling. *Semantic routing* [23] classifies the query by intent and directs it to an appropriate retriever, metadata filter, or bypass action before any similarity computation occurs (Figure 2).

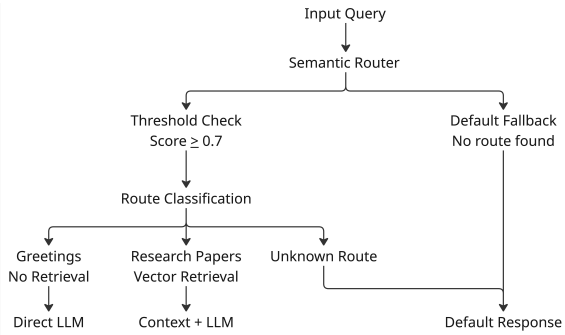


Figure 2. *Semantic routing assigns queries to domain-specific retrievers or bypass actions based on intent classification before vector search begins.*

Each route is defined by example phrases and a confidence threshold. Queries that fall below every threshold route to a general retriever. Routing is a pre-retrieval optimization because it selects the retrieval index (or knowledge source) and retrieval configuration upstream of search itself, preventing both wasted computation on out-of-scope inputs and imprecise matches from searching the wrong knowledge domain. In agent systems where retrieval is one tool among several, the same intent signals can determine whether to invoke retrieval at all and which retriever to call.

5. Mid-Retrieval Optimization

Once documents and queries are prepared, mid-retrieval optimization controls the matching function: given a query and an indexed corpus, which chunks should be returned, in what order, and at what granularity? The idea of retrieval granularity is discussed later in the context of Parent-Child Expansion. The techniques below are independent components; each can be enabled, disabled, or reordered without adopting a fixed end-to-end pipeline.

5.1. Similarity Search and Diversity

Cosine similarity between query and chunk embeddings is the default retrieval function:

$$\text{sim}(\boldsymbol{q}, \boldsymbol{c}_i) = \frac{\boldsymbol{q} \cdot \boldsymbol{c}_i}{\mid \boldsymbol{q} \mid \mid \boldsymbol{c}_i \mid} \tag{1}$$

where $\boldsymbol{q}$ is the query embedding, $\boldsymbol{c}_i$ is the embedding of the i-th chunk.

Top-k selection returns the k chunks with the highest similarity scores, where the score is typically computed using cosine similarity (or another similarity metric) between the query embedding and each chunk embedding. The value of k is an application-dependent hyperparameter that is typically tuned empirically to balance retrieval recall against context-window constraints rather than being computed automatically. This works well when query and document language align semantically, but corpora with repetitive or overlapping content often yield near-duplicate results that use up context window capacity without adding information. Maximal Marginal Relevance (MMR) [24] addresses redundancy by iteratively selecting chunks that maximize query relevance while minimizing similarity to previously selected chunks:

$$\text{MMR} = \arg \max_{c_i \in C \setminus S} \lambda \cdot \text{sim}(\boldsymbol{q}, \boldsymbol{c}_i) - (1 - \lambda) \cdot \max_{c_j \in S} \text{sim}\big(\boldsymbol{c}_i, \boldsymbol{c}_j\big) \tag{2}$$

where C is the set of candidate chunks returned by the initial retrieval stage, S is the set of chunks already selected, $\boldsymbol{c}_j$ is a previously selected chunk.

Here, $\lambda \in [0, 1]$ controls the trade-off between relevance and diversity: lower values favor more diverse retrieval, while higher values prioritize relevance to the query.

5.2. Retrieval Filtering

Two lightweight filters constrain what search returns before ranking. *Score thresholding* drops candidates below a minimum similarity θ, returning fewer than k results when nothing clears the cutoff, preferable to passing weak matches to the generator. *Metadata filtering* restricts search to chunks tagged by source, date, topic, or chunk type when scope is already known from the query or an upstream route.

5.3. Parent-Child Expansion

Parent-child expansion decouples search granularity from the unit passed to the generator. The common approach is two-step: similarity search runs over child embeddings, then each match is mapped through stored `parent_id` metadata to a larger parent chunk in a document store; retrieval finds the precise span, but generation sees the surrounding section. Implementations vary in the expansion rule. Some always return parents after child retrieval. While the standard implementation stores embeddings only for child chunks, some hierarchical retrieval systems additionally maintain parent embeddings and apply a secondary ranker to choose parent or child output based on query specificity. A lightweight alternative uses retrieval confidence, keeping child chunks when top-k scores are strong and expanding to parents when scores signal a partial or context-dependent match. The component plugs into any retrieval path that already indexes hierarchical chunks (Table 2).

5.4. Hybrid Search

Dense retrieval captures paraphrases and conceptual similarity but misses exact keywords, acronyms, and rare terms; BM25 captures those lexical signals but not meaning. *Hybrid retrieval* runs both in parallel and merges the ranked lists. Weighted score fusion is one option; *reciprocal rank fusion* (RRF) [25] is a rank-agnostic alternative that needs no score normalization:

$$\mathrm{RRF}(d) = \sum_{L \in \mathcal{L}} \frac{1}{k + \mathrm{rank}_L(d)} \tag{3}$$

where d is a retrieved document (or chunk), $\mathcal{L}$ is the set of ranked retrieval lists being fused (e.g., dense retrieval and BM25), $\mathrm{rank}_L(d)$ is the rank of d in list L, and k is a positive smoothing constant that reduces the influence of very high-ranked documents.

Documents appearing in multiple lists rank higher. The same fusion step applies when pre-retrieval query expansion produces several search variants: each variant yields a ranked list, and RRF merges them into one candidate pool without prescribing how those queries were generated.

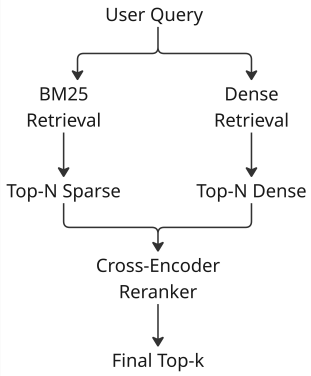


Figure 3. *Hybrid retrieval combines sparse keyword matching (BM25) with dense embedding search, fusing candidates before final ranking.*

5.5. Reranking

Bi-encoder retrieval encodes queries and documents independently, enabling efficient approximate nearest-neighbor search but limiting token-level interactions. *Cross-encoder reranking* [11] scores query–document pairs jointly, improving precision at the cost of latency. The standard pattern retrieves a larger candidate set ($k' \gg k$), reranks with a cross-encoder, and returns the top-k. Reranking composes with hybrid search or RRF output: fusion produces the candidate pool, reranking orders it. Model choice trades quality against speed i.e. full cross-encoders for accuracy, lighter models for low-latency filtering.

Table 5. *Mid-retrieval methods, targets, and latency cost.*

Method	Primary Target	Adds Latency?
Cosine top-k	Baseline relevance	No
MMR	Retrieval noise (redundancy)	Low
Retrieval filtering	Retrieval noise; wrong scope	No
Parent-child expansion	Context fragmentation	Low
Hybrid (BM25 + dense)	Query mismatch (terms vs. paraphrase)	Medium
Cross-encoder reranking	Context precision	High

6. Post-Retrieval Optimization

Retrieved chunks are rarely ready for direct use by a language model. Post-retrieval optimization prepares the ranked result set by filtering noise, tightening passages, and assembling context before generation or before control returns to an orchestrating agent. The components below are independent; each plugs in after retrieval and reranking without requiring a multistep process at the post-retrieval stage.

6.1. Context Preparation

Reranking improves ordering but does not remove every weak passage or irrelevant sentence within a relevant document. Three post-retrieval components address what remains.

Retrieval grading [13] applies a lightweight relevance judge to each retrieved chunk, labeling it relevant, irrelevant, or ambiguous before it enters the prompt. Chunks below confidence are dropped or downweighted, targeting semantic false positives that survive similarity search and reranking. This extracts the evaluator from corrective RAG without its web-search fallback or query-rewrite loop.

Knowledge refinement [13] operates on chunks that pass grading: each passage is split into sentences, unrelated sentences are removed, and the remainder is reassembled into a tighter unit. Where grading removes whole chunks, refinement strips filler within chunks that are partially on-topic.

Context compression [19] reduces the amount of text passed to the generator by retaining only the information required to answer the current query. Unlike knowledge refinement, which removes irrelevant content while preserving the remaining passage, compression may further shorten or summarize relevant content to fit the available context window. Implementations include extractive selection, embedding-based span selection, and abstractive summarization.

Retrieval grading filters out irrelevant chunks, knowledge refinement removes irrelevant content within retained chunks, and context compression reduces the remaining context to its query-relevant portions. These components are complementary and can be applied sequentially.

Table 6. *Post-retrieval context preparation components.*

Component	Primary Target	Adds Latency?
Retrieval grading	Retrieval noise (false-positive chunks)	Low–Medium
Knowledge refinement	Retrieval noise (within-chunk filler)	Medium
Context compression	Retrieval noise; context budget	Medium

6.2. Document Assembly

Production RAG concatenates the prepared retrieved set into a single prompt and generates one answer (*stuff* in LangChain terminology [19]). Larger context windows and stronger models have made this the default; earlier multi-pass strategies (refine, map-reduce, map-rerank) addressed context limits that are rarely a concern today. In agent workflows, the same pattern applies: retrieved context is formatted as structured input for the orchestrating model's next step. For multimodal corpora, assembly must preserve chunk type (SQL results as structured text, image references alongside narrative) so the generator receives each modality in a usable form.

7. Conclusion and Future Work

Retrieval quality is the foundation of effective RAG systems, yet most implementations treat it as an afterthought. We presented a three-stage framework: preparing documents and queries, controlling search and ranking, and refining context before generation, that maps modular components to three recurring failure modes: context fragmentation, query mismatch, and retrieval noise. Effective systems compose techniques across these stages rather than relying on any single fix. Whether retrieval runs once in a standalone pipeline or repeatedly as an agent tool [2], each invocation passes through the same stages; orches-

tration decides when to retrieve, while the framework specifies what each call should return.

Several directions remain open. Systematic evaluation of component combinations rather than isolated techniques would quantify how pre-, mid-, and post-retrieval optimizations compose under fixed embedding and generation settings. Text-centric benchmarks do not fully capture multimodal chunking and typed retrieval; corpora with tables, figures, and layout structure need dedicated evaluation. Future work should also study adaptive pre-retrieval: selecting chunking strategy based on document type: plain text versus layout-rich or multimodal sources and routing query enhancement through agentic knowledge, where an orchestrator chooses expansion, decomposition, or rewriting based on query intent and conversation context rather than applying a fixed transformation pipeline. As context windows grow, the bottleneck shifts from fitting retrieved content to ensuring the right content reaches the model; engineering retrieval at the stage where each problem originates remains the distinguishing factor between demonstration and production systems.

References

[1] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020, [Online]. Available: https://arxiv.org/abs/2005.11401

[2] A. Singh, A. Ehtesham, S. Kumar, and T. T. Khoei, "Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG," *arXiv preprint arXiv:2501.09136*, 2025, [Online]. Available: https://arxiv.org/abs/2501.09136

[3] LangChain AI, "LangGraph: Build resilient language agents as graphs." [Online]. Available: https://github.com/langchain-ai/langgraph

[4] Z. Wang *et al.*, "Document Segmentation Matters for Retrieval-Augmented Generation," in *Findings of the Association for Computational Linguistics: ACL 2025*, Vienna, Austria, 2025, pp. 8063–8075. doi: 10.18653/v1/2025.findings-acl.422.

[5] Y. Wang, H. Zhang, L. Pang, B. Guo, H. Zheng, and Z. Zheng, "MaFeRw: Query Rewriting with Multi-Aspect Feedbacks for Retrieval-Augmented Large Language Models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025, pp. 25434–25442. doi: 10.1609/aaai.v39i24.34732.

[6] Z. Wu *et al.*, "CONQRR: Conversational Query Rewriting for Retrieval with Reinforcement Learning," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, Abu Dhabi, United Arab Emirates, 2022, pp. 10000–10014. doi: 10.18653/v1/2022.emnlp-main.679.

[7] I. S. Singh *et al.*, "ChunkRAG: A Novel LLM-Chunk Filtering Method for RAG Systems," *arXiv preprint arXiv:2410.19572*, 2024, [Online]. Available: https://arxiv.org/abs/2410.19572

[8] V. Karpukhin *et al.*, "Dense Passage Retrieval for Open-Domain Question Answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781. [Online]. Available: https://arxiv.org/abs/2004.04906

[9] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009, doi: 10.1561/1500000019.

[10] O. Khattab and M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020, pp. 39–48. [Online]. Available: https://arxiv.org/abs/2004.12832

[11] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *arXiv preprint arXiv:1908.10084*, 2019, [Online]. Available: https://arxiv.org/abs/1908.10084

[12] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2310.11511

[13] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective Retrieval Augmented Generation," *arXiv preprint arXiv:2401.15884*, 2024, [Online]. Available: https://arxiv.org/abs/2401.15884

[14] M. Arora, "Research Papers Collection from arXiv." [Online]. Available: https://huggingface.co/datasets/mahimaarora025/research_papers

[15] Qdrant Team, "Qdrant: Vector Database for the Next Generation of AI Applications." [Online]. Available: https://qdrant.tech/

[16] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," *arXiv preprint arXiv:2309.15217*, 2023, [Online]. Available: https://arxiv.org/abs/2309.15217

[17] M. Arora and A. Jha, "Retrieval Playground: A Python Toolkit for RAG Experimentation and Evaluation." [Online]. Available: https://github.com/mahimaarora/retrieval-playground

[18] N. Livathinos *et al.*, "Docling: An Efficient Open-Source Toolkit for AI-driven Document Conversion," *arXiv preprint arXiv:2501.17887*, 2025, [Online]. Available: https://arxiv.org/abs/2501.17887

[19] LangChain AI, "LangChain: Building context-aware reasoning applications." [Online]. Available: https://github.com/langchain-ai/langchain

[20] Anthropic, "Contextual Retrieval in AI Systems." [Online]. Available: https://www.anthropic.com/engineering/contextual-retrieval

[21] L. Gao, X. Ma, J. Lin, and J. Callan, "Precise Zero-Shot Dense Retrieval without Relevance Labels," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Toronto, Canada, 2023, pp. 1762–1777. doi: 10.18653/v1/2023.acl-long.99.

[22] H. S. Zheng *et al.*, "Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://arxiv.org/abs/2310.06117

[23] Aurelio Labs, "Semantic Router: Intelligent Query Routing for LLM Applications." [Online]. Available: https://github.com/aurelio-labs/semantic-router

[24] J. Carbonell and J. Goldstein, "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries," in *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1998, pp. 335–336. doi: 10.1145/290941.291025.

[25] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759. doi: 10.1145/1571941.1572114.